

CTU-Chat: Asymmetric Multi-Agent Agentic RAG with Knowledge Graph for Vietnamese University Counseling

Cao Tuong Hung¹

Can Tho University, Can Tho, Vietnam
hungb2303873@student.ctu.edu.vn

Abstract. University student-service chatbots must accurately handle diverse query types—from relational academic-program lookups to exact tuition calculations and open-ended policy questions—yet existing Retrieval-Augmented Generation (RAG) systems treat all queries uniformly, wasting computation on simple questions and lacking precision on complex ones. This paper presents **CTU-Chat**, a Vietnamese university counseling system built on three novel contributions: (i) an *Asymmetric Multi-Agent* architecture that assigns differentiated autonomy levels—full ReAct reasoning, context-bounded ReAct, or zero-shot chain—to specialist agents based on query complexity; (ii) *Intent-Guided Subspace Retrieval Routing* (IG-SRR), which couples LLM intent classification with deterministic metadata lane filters to confine hybrid retrieval (BM25 + dense vector + RRF + cross-encoder reranking) to ontologically correct document subspaces; and (iii) a *Neo4j Knowledge Graph* with typed Python tools that eliminates schema hallucination by replacing raw Cypher generation with safe, parameterized graph queries. The system is implemented with LangGraph, Qdrant, Neo4j, and Gemini, and evaluated on a 100-question Vietnamese benchmark spanning four domains. Preliminary results show 91% LLM-judge pass rate, 96.67% tool-calling accuracy, and 100% expected-source coverage across reranker variants. Controlled retrieval ablations and RAGAS-based answer-quality evaluations are in progress.

Keywords: Retrieval-Augmented Generation · Multi-Agent Systems · Knowledge Graph · Vietnamese NLP · University Chatbot · LangGraph

1 Introduction

The rapid expansion of higher education enrollment in Vietnam has placed unprecedented pressure on university administrative offices to respond to student inquiries regarding academic programs, tuition fees, scholarships, and institutional policies [1]. Can Tho University (CTU), the largest institution in the Mekong Delta region, serves tens of thousands of students across more than

100 academic programs, generating a constant stream of diverse counseling requests. Traditional FAQ-based or rule-based chatbots have proven inadequate: they cannot reason over relational academic structures, perform precise financial calculations, or adapt to annually changing regulations.

Retrieval-Augmented Generation (RAG) [1] has emerged as the dominant paradigm for grounding large language models (LLMs) in external knowledge. Recent advances have extended basic RAG along two axes. First, *hybrid retrieval* pipelines combine sparse lexical search (BM25 [2]) with dense vector embeddings and reciprocal-rank fusion (RRF [3]) to capture both keyword precision and semantic similarity. Second, *Agentic RAG* integrates autonomous agents capable of tool use, multi-step reasoning, and dynamic planning into the retrieval-generation loop. Knowledge Graph RAG (GraphRAG) [4] further augments this paradigm by leveraging graph-structured knowledge for relational queries that flat document retrieval cannot answer.

Despite these advances, deploying Agentic GraphRAG systems in domain-specific educational settings reveals five critical research gaps identified in recent literature:

1. **Schema hallucination and tool overload:** When LLM agents generate raw Cypher queries against knowledge graphs, they frequently produce syntactically valid but semantically incorrect queries, especially on complex schemas with many relationship types [4].
2. **Latency and computational overhead:** Homogeneous multi-agent architectures that assign full ReAct reasoning loops to every query—including trivial ones—consume excessive LLM tokens and introduce unacceptable latency for real-time counseling [5].
3. **Score-space incompatibility in hybrid fusion:** BM25 scores, dense cosine similarities, and graph topology signals occupy fundamentally different mathematical spaces, making naïve fusion via static RRF suboptimal.
4. **Temporal knowledge degradation:** Static knowledge graphs cannot track annually changing tuition rates, admission criteria, or scholarship policies without costly full recomputation.
5. **Lack of multi-agent evaluation frameworks:** Standard end-result metrics (BLEU, ROUGE, or single-turn RAGAS) are blind to agent coordination quality, tool-call validity, and routing accuracy.

This paper addresses Gaps 1–3 directly and proposes partial solutions for Gap 5. Specifically, we present **CTU-Chat**, a Vietnamese university counseling system with three core contributions:

- An **Asymmetric Multi-Agent** architecture implemented via LangGraph’s Supervisor Pattern, where each specialist agent receives a differentiated autonomy level matched to the inherent complexity of its domain (Section 3.2).
- **Intent-Guided Subspace Retrieval Routing (IG-SRR)**, a mechanism that couples LLM-based intent classification with deterministic metadata lane filters, confining hybrid BM25 + dense retrieval to ontologically correct document subspaces before RRF fusion (Section 3.3).

- **Typed Knowledge Graph tools** for Neo4j that replace unconstrained Cypher generation with safe, parameterized Python functions, eliminating schema hallucination at the architectural level (Section 3.5).

The system is built with FastAPI, LangGraph, Qdrant vector database, Neo4j graph database, Redis session memory, PostgreSQL persistent storage, and Google Gemini LLMs. We evaluate on a curated 100-question Vietnamese benchmark balanced across four domains: tuition, exemption/social-support, scholarships, and student loans.

The remainder of this paper is organized as follows. Section 2 reviews related work on RAG, knowledge graphs, and multi-agent systems, identifying the research gaps that motivate our design. Section 3 presents the proposed model in detail, including system architecture, agent design, retrieval pipeline, and routing algorithms. Section 4 describes the experimental setup, evaluation metrics, and results. Section 5 concludes the paper and outlines future directions.

2 Related Work

2.1 Retrieval-Augmented Generation and Hybrid Search

Lewis et al. [1] introduced RAG as a method to ground LLM outputs in retrieved documents, significantly reducing hallucination. The standard dense retrieval pipeline encodes queries and documents into a shared vector space and retrieves by cosine similarity. However, dense retrieval alone struggles with exact keyword matches, abbreviations, and Vietnamese compound nouns.

BM25 [2] addresses these shortcomings through sparse lexical matching with term-frequency saturation and document-length normalization. Hybrid retrieval systems combine both approaches. Reciprocal Rank Fusion (RRF) [3] merges ranked lists without requiring score calibration:

$$s_{\text{RRF}}(d) = \sum_{r \in \{\text{dense}, \text{BM25}\}} \frac{1}{k + \text{rank}_r(d)} \quad (1)$$

where k is a smoothing constant (typically 60). Cross-encoder rerankers [7] then refine the fused list by jointly encoding query–document pairs.

While hybrid search improves recall, applying it over the *entire* corpus introduces noise from semantically similar but contextually irrelevant documents—a problem our IG-SRR mechanism directly addresses.

2.2 Knowledge Graphs in RAG Systems

GraphRAG [4] extends RAG by constructing knowledge graphs from document corpora and using graph traversal for multi-hop reasoning. KA-RAG [8] integrates knowledge graphs with agentic retrieval for improved factual grounding. However, these systems typically require the LLM to generate Cypher or SPARQL queries directly, leading to schema hallucination when the graph

schema is complex. CyVerACT [9] proposes verification workflows for Cypher translation but adds latency through multiple validation rounds.

Our approach avoids this problem entirely by *encapsulating* graph queries within typed Python tool functions, exposing only domain-relevant parameters to the agent.

2.3 Multi-Agent Systems and Agentic AI

The ReAct framework [5] enables LLM agents to interleave reasoning traces with tool actions. LangGraph extends this to multi-agent workflows with shared state machines. Existing multi-agent architectures (AutoGen, CrewAI) typically employ *homogeneous* agent designs where every node possesses full planning, reasoning, and tool-use capabilities. While maximally flexible, this approach ignores the marginal cost of cognitive operations: a simple FAQ lookup does not warrant a multi-step ReAct loop.

Recent benchmarks [11] demonstrate that supervisor-based architectures outperform fully autonomous agent meshes on structured tasks, supporting our asymmetric design philosophy.

2.4 Agentic RAG in Education

Ma et al. [6] proposed REBot, combining RAG with CatRAG for semantic enrichment and graph routing in educational contexts. An empirical study of multi-agent RAG for real-world university applications [10] demonstrated that domain-specialized agents outperform monolithic RAG systems on diverse query types. However, neither system addresses the Vietnamese language context or proposes intent-guided retrieval subspace isolation.

2.5 Research Gaps and Research Questions

Table 1 summarizes the five research gaps identified from the literature and maps them to our proposed solutions.

Based on these gaps, we formulate three research questions:

- RQ1:** Does an asymmetric multi-agent architecture reduce computational cost compared to homogeneous agent designs while maintaining answer quality?
- RQ2:** Does Intent-Guided Subspace Retrieval Routing (IG-SRR) improve retrieval precision compared to global corpus search?
- RQ3:** Does hybrid RAG combined with a knowledge graph outperform single-mode retrieval for domain-specific Vietnamese university counseling?

3 Proposed Model

3.1 System Architecture Overview

CTU-Chat is a multi-agent system built on FastAPI and LangGraph, integrating four data stores, two LLM configurations, and four specialist agents. Figure 1 presents the overall system architecture.

Table 1: Research gaps and proposed solutions in CTU-Chat.

Gap	Description	CTU-Chat Solution	Status
G1	Schema hallucination NL→Cypher	in Typed Python tools; no raw Cypher	Addressed
G2	ReAct overhead on queries	simple Asymmetric agent levels	autonomy Addressed
G3	BM25/Dense/Graph space mismatch	score- IG-SRR: lane-bounded space fusion	sub- Addressed
G4	Temporal degradation static KGs	on Academic-year metadata ter (partial)	fil- Partial
G5	Lack of multi-agent evalua- tion	LLM judge + tool testing + ablation	Partial

Table 2: Asymmetric agent design: each agent receives autonomy proportional to its domain complexity.

Agent	Type	Tools	Data Source	Via Retrieval?
Academic	ReAct (full)	6 Neo4j tools	Knowledge Graph	No
Financial	ReAct (context-bounded)	3 tools	RAG + Neo4j	Yes
Scholarship	ReAct (context-bounded)	1 tool	RAG context	Yes
General	Zero-shot chain	None	RAG context	Yes

The infrastructure layer comprises four Docker-containerized services:

- **Qdrant** (v1.18.1): Vector database storing 768-dimensional child-chunk embeddings with cosine distance for dense retrieval.
- **Redis** (v7): In-memory cache for recent conversation history (bounded to 5 messages for context, 50 for session).
- **PostgreSQL** (v15): Relational storage for users, sessions, long-term chat history, parent documents, and metadata.
- **Neo4j** (v5.20): Graph database modeling academic programs, courses, prerequisites, and tuition structures.

The primary LLM is `gemini-3.5-flash-lite` for supervisor routing and all agent generation. A secondary `gemini-3.1-flash-lite` model handles query rewriting for lower latency and cost.

3.2 Asymmetric Multi-Agent Design

A central contribution of CTU-Chat is its *asymmetric* allocation of agent autonomy. Unlike homogeneous architectures where every agent runs a full ReAct loop, we match cognitive complexity to query requirements:

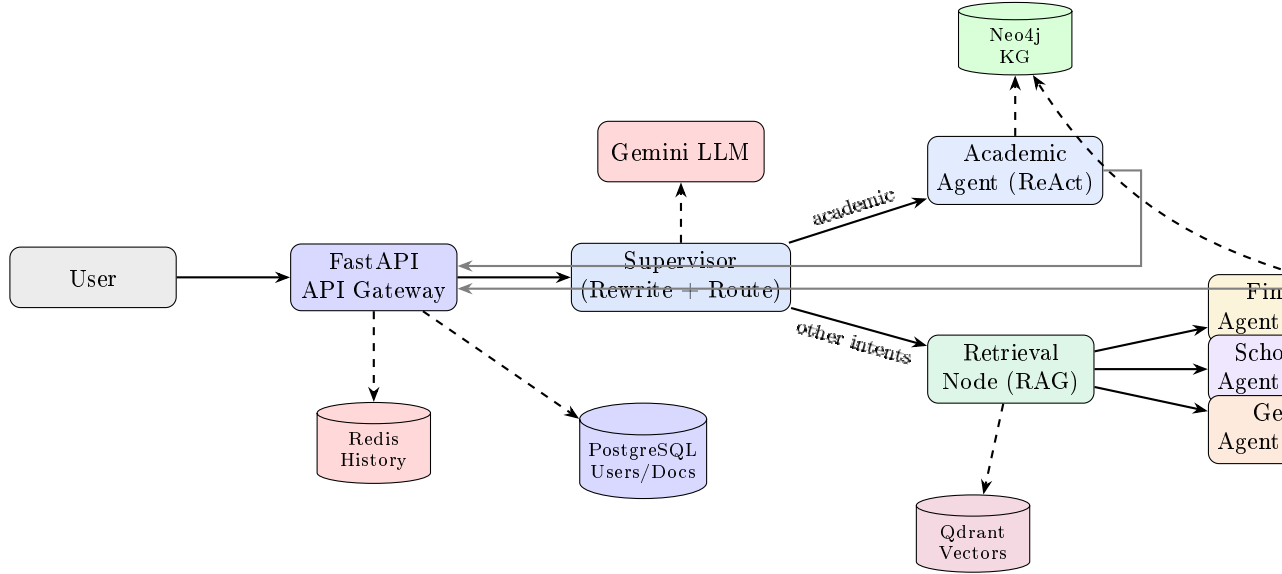


Fig. 1: CTU-Chat system architecture. The Supervisor routes queries to either the Academic Agent (direct Neo4j access) or the Retrieval Node (hybrid RAG), which then dispatches context to domain-specific agents. Dashed lines indicate data-store connections.

Academic Agent. This is the only fully autonomous agent. It operates a ReAct loop with six typed Neo4j tools: program lookup, program comparison, criteria-based search, prerequisite chain traversal, shared-course identification, and reverse course lookup. It bypasses document retrieval entirely, directly querying the knowledge graph through parameterized Cypher functions. This design exploits the natural isomorphism between graph traversal and the sequential reasoning pattern of LLMs.

Financial Agent. This agent receives pre-retrieved RAG context from the Retrieval Node (filtered by IG-SRR lanes) and has access to three tools: graph-based tuition lookup, tuition regulation query, and a deterministic tuition calculator. The pre-loaded context constrains the agent’s search space, preventing number hallucination on financial data while still allowing multi-step reasoning for complex calculations (e.g., “calculate net tuition after 70% exemption for Law program”).

Scholarship Agent. Similar to the Financial Agent but with a single computational tool for GPA/conduct-based scholarship calculation. Its bounded autonomy reflects the simpler decision space of scholarship queries.

General Agent. Implemented as a simple `prompt | LLM | StrOutputParser()` chain with *zero* tools and *no* reasoning loop. This agent handles open-ended

policy questions where a single LLM call over retrieved context suffices. By eliminating the ReAct overhead (3–5 LLM calls per query), this agent achieves minimal latency for the most common query type.

This asymmetric design applies the Principle of Least Privilege to cognitive resources: agents receive only the autonomy they need, reducing token consumption and latency while maintaining answer quality where complex reasoning is required.

3.3 Intent-Guided Subspace Retrieval Routing (IG-SRR)

The second core contribution is IG-SRR, a coupled mechanism that unifies intent classification with retrieval space partitioning. The Supervisor performs two decisions in a *single* LLM call via structured output:

$$f_{\text{route}}(q, h) \rightarrow (\text{next_agent}, \text{intent}) \quad (2)$$

where q is the (potentially rewritten) query, h is the bounded conversation history, `next_agent` $\in \{\text{academic}, \text{financial}, \text{scholarship}, \text{general}\}$ selects the processing agent, and `intent` $\in \{\text{actual_tuition}, \text{exemption_basis}, \text{exemption_policy}, \text{calculation}, \text{scholarship}, \text{student_loan}, \text{academic_program}, \dots\}$ selects the *document subspace*.

Each intent maps to one or more *Deterministic Retrieval Lanes*—metadata filter specifications that constrain both Qdrant dense search and BM25 sparse search:

$$\mathcal{A}_l(q) = \{e \mid \text{status}(e) = \text{active}, \text{domain}(e) = d_l, \text{kind}(e) = k_l, \text{fee}(e) = f_l\} \quad (3)$$

For example, the intent `calculation` activates three concurrent lanes: `actual_tuition`, `exemption_basis`, and `exemption_policy`, ensuring the agent receives all document types needed for a complete financial computation. This one-to-many intent-to-lane mapping is a key novelty: the LLM predicts the *nature* of required documents rather than extracting named entities for filtering.

Table 3 shows the complete intent-to-lane mapping.

3.4 Hybrid Retrieval Pipeline

The retrieval pipeline operates *within* the subspace defined by IG-SRR lanes:

Step 1: Document Ingestion and Parent-Child Chunking. Source documents (Markdown from manual curation or LlamaParse OCR) are split into parent sections (2,800-character limit, 100-character overlap) and child chunks (400 characters, 50-character overlap). A table-aware splitter repeats headers in every child chunk to preserve column semantics. Parent content is stored in PostgreSQL; child embeddings (768-dimensional, from `bkai-foundation-models/vietnamese-bi-encoder`) are indexed in Qdrant.

Table 3: Intent-to-lane mapping in the IG-SRR mechanism.

Intent	Lanes Activated	Documents Retrieved
actual_tuition	actual_tuition	Actual tuition rate tables
exemption_basis	exemption_basis	Exemption calculation bases
exemption_policy	exemption_policy	Exemption policy documents
calculation	actual + basis + policy	All three for computation
both / ambiguous	actual + basis	Both rate tables
scholarship	scholarship	Scholarship documents
student_loan	student_loan	Student loan documents
academic_program	(empty)	Uses Neo4j tools directly
other	default (no filter)	Global search

Step 2: Lane-Filtered Hybrid Search. For a given query q and lane set L :

- **Dense search:** Qdrant retrieves child chunks by cosine similarity, filtered by lane metadata.
- **Sparse search:** BM25 (with PyVi Vietnamese tokenization, $k_1 = 1.5$, $b = 0.75$) retrieves children, filtered by the same lane metadata.
- **Parent aggregation:** Children are grouped by parent ID; the maximum child score represents each parent.

Step 3: RRF Fusion. Parent lists from dense and sparse branches are fused using Equation 1 with $k = 60$.

Step 4: Cross-Encoder Reranking with Temporal Tie-Break. The BAAI/bge-reranker-v2-m3 cross-encoder rescores the top 15 fused parents. Within a score tolerance of 0.05, documents are reordered by timestamp (newer first), implementing a bounded temporal tie-break that preserves source coverage without degrading relevance ranking.

3.5 Knowledge Graph Integration

The Neo4j knowledge graph models the CTU academic domain with nodes for Programs, Courses, and structural relationships (prerequisites, electives, shared courses). Rather than exposing the graph schema to the LLM for raw Cypher generation, we implement six **typed Python tool functions**:

1. `tra_cuu_nganh`: Look up detailed program information
2. `so_sanh_nganh`: Compare two programs side-by-side
3. `tim_nganh`: Search programs by criteria
4. `xem_chuoi_tien_quyet`: Traverse prerequisite chains
5. `mon_chung_giua_nganh`: Find shared courses between programs
6. `tim_nganh_co_mon`: Reverse lookup: which programs teach course X

Algorithm 1 Supervisor Node: Query Rewrite and Routing**Require:** Query q , chat history h , rewrite LLM M_r , routing LLM M_s **Ensure:** Updated state with `search_query`, `next_agent`, `intent`

```

1: function SUPERVISORNODE( $q, h$ )
2:    $q_s \leftarrow q$                                  $\triangleright$  Default: use original query
3:   if SHOULDREWRITE( $q, h$ ) then                   $\triangleright$  Contains pronouns, ellipsis
4:      $q_r \leftarrow M_r(q, h[-1])$                  $\triangleright$  Rewrite with previous turn
5:     if VALIDATEREWRITE( $q_r, q$ ) then             $\triangleright$  No new entities/numbers
6:        $q_s \leftarrow q_r$ 
7:   (agent, intent)  $\leftarrow M_s$ .structured_output( $q_s, h$ )
8:   return {search_query:  $q_s$ , next_agent: agent, intent: intent}

```

Each tool encapsulates a validated Cypher query with parameterized inputs, eliminating the risk of schema hallucination. The Financial Agent additionally accesses two graph tools (`tra_cuu_hoc_phi_graph` for tuition lookup and `tra_cuu_quy_dinh_hoc_phi` for fee regulations) and one deterministic calculator (`tin_h_toan_hoc_phi`).

3.6 Query Processing Algorithm

Algorithm 1 describes the Supervisor routing logic, which performs query rewriting and intent-based routing in a unified pipeline.

Algorithm 2 formalizes the IG-SRR lane selection and retrieval process.

3.7 System Activity Flow

Figure 2 presents the complete activity flowchart of CTU-Chat, showing the decision points where the system branches based on query type, intent classification, and data source selection.

The flowchart illustrates several key design decisions:

- **Academic queries bypass retrieval entirely**, going directly to the Neo4j-powered ReAct agent—this eliminates unnecessary vector search latency for relational queries.
- **Financial queries receive both graph-structured context and RAG documents**, ensuring the agent has exact figures from Neo4j alongside explanatory policy text from the document corpus.
- **General queries follow the shortest path**: retrieval $\rightarrow$ zero-shot chain, requiring only two LLM calls total (one for routing, one for generation).

Data Characteristics and Retrieval Strategy Justification. The choice of retrieval modality is driven by the inherent structure of CTU data:

- **Academic programs** (curricula, prerequisites, shared courses): inherently *relational* data with graph topology $\rightarrow$ Neo4j Knowledge Graph with graph traversal tools.

Algorithm 2 Intent-Guided Subspace Retrieval Routing (IG-SRR)**Require:** Search query q_s , routing decision ($agent, intent$)**Ensure:** Retrieved context C , retrieval instruction I

```

1: function RETRIEVALNODE( $q_s, agent, intent$ )
2:   if  $agent = \text{"academic"}$  then
3:     return  $\emptyset$  ▷ Academic agent uses Neo4j directly
4:    $L \leftarrow \text{BUILDLANES}(intent)$  ▷ Intent  $\rightarrow$  lane set
5:    $C_{\text{struct}} \leftarrow \emptyset$ 
6:   if  $agent = \text{"financial"}$  then
7:      $C_{\text{struct}} \leftarrow \text{NEO4JLOOKUP}(q_s)$  ▷ Graph structured data
8:     if  $C_{\text{struct}} = \emptyset$  then
9:        $C_{\text{struct}} \leftarrow \text{JSONCATALOGFALLBACK}(q_s)$ 
10:  for all lane  $l \in L$  do
11:     $D_l \leftarrow \text{DENSESEARCH}(q_s, \text{filter}(l))$  ▷ Qdrant
12:     $S_l \leftarrow \text{SPARSESEARCH}(q_s, \text{filter}(l))$  ▷ BM25
13:     $P_l \leftarrow \text{PARENTAGGREGATE}(D_l \cup S_l)$ 
14:   $F \leftarrow \text{RRFFUSION}(\bigcup_l P_l)$  ▷ Eq. 1
15:   $R \leftarrow \text{RERANK}(F, q_s)$  ▷ Cross-encoder + tie-break
16:   $C \leftarrow C_{\text{struct}} \cup \text{BUILDCONTEXT}(R)$ 
17:   $I \leftarrow \text{BUILDINSTRUCTION}(intent, L)$ 
18:  return  $(C, I)$ 

```

- **Tuition and fees:** *tabular* data requiring numerical precision $\rightarrow$ Hybrid RAG with lane-filtered structured context + deterministic calculation tools.
- **Policies and regulations:** *narrative* text requiring semantic understanding $\rightarrow$ Dense vector search for semantic similarity + BM25 for keyword matching on Vietnamese compound nouns.
- **Scholarships and loans:** mix of *policy text* and *computational rules* $\rightarrow$ RAG context + specialized calculation tools.

This analysis justifies when to use keyword search (BM25: exact terms, regulation codes), when to use semantic search (Dense: paraphrased questions, synonyms), and when to use graph queries (Neo4j: multi-hop relational lookups).

4 Experimental Results

4.1 Experimental Dataset

We constructed a Vietnamese-language evaluation dataset of 100 questions balanced across four domains: exemption and social support (25), tuition (25), student loans (25), and scholarships (25). Each question includes:

- The **question text** in Vietnamese, reflecting real student queries collected from CTU counseling channels and FAQ pages.

Table 4: Experimental tools and versions.

Component	Implementation
Web Framework	FastAPI
Agent Orchestration	LangGraph (StateGraph)
Dense Vector Store	Qdrant v1.18.1 (768-dim, cosine)
Sparse Retrieval	BM25s + PyVi tokenizer
Graph Database	Neo4j v5.20.0
Cross-Encoder Reranker	BAAI/bge-reranker-v2-m3
Session Store	Redis v7
Persistent Store	PostgreSQL v15
Primary LLM	Gemini gemini-3.5-flash-lite
Rewrite LLM	Gemini gemini-3.1-flash-lite
Embedding Model	bkai-foundation-models/vietnamese-bi-encoder
Evaluation Framework	RAGAS + custom LLM judge

- An **expected answer** with key factual elements that a correct response must contain.
- The **expected source filename(s)** identifying which document(s) should be retrieved.
- The **domain label** for stratified analysis.

The corpus comprises 50 Markdown policy and tuition documents under `data/markdown` and 114 academic-program Markdown files under `data/markdown_graph`. The active Qdrant collection contains 1,303 vector points. Documents were curated from official CTU publications, with metadata including domain, content kind, fee kind, academic year, and active/inactive status.

4.2 Experimental Tools

Table 4 summarizes the software stack used in experiments.

4.3 Evaluation Metrics

We employ three categories of metrics:

Retrieval Metrics. Precision at 5 (P@5), Recall at 5 (Recall@5), and Mean Reciprocal Rank (MRR):

$$P@5 = \frac{|\text{Rel} \cap \text{Top5}|}{5}, \quad \text{Recall@5} = \frac{|\text{Rel} \cap \text{Top5}|}{|\text{Rel}|}, \quad \text{MRR} = \frac{1}{N} \sum_{q=1}^N \frac{1}{\text{rank}_q^{\text{first}}} \quad (4)$$

Answer Quality Metrics (RAGAS). Answer Relevance, Context Recall, Context Precision, and Answer Correctness, evaluated using an LLM-as-a-judge approach with the judge model and prompt configuration reported alongside results.

Table 5: Scenario 1: Retrieval ablation—isolating dense, sparse, hybrid, and reranked retrieval. All values are pending controlled experiments.

Configuration	P@5	R@5	MRR	Ans.Rel	Ctx.Rec	Ctx.Prec	Ans.Corr
Dense only	–	–	–	–	–	–	–
Sparse only (BM25)	–	–	–	–	–	–	–
Hybrid RRF	–	–	–	–	–	–	–
Hybrid RRF + Rerank	–	–	–	–	–	–	–

Table 6: Scenario 2: Graph and agent ablation—evaluating routing accuracy, tool validity, and graph query quality. All values are pending.

Variant	Route	Acc.	Tool Valid.	Graph Metric	Answer Metrics
Graph only	n/a	–	–	–	–
Fixed route + Hybrid	–	–	–	n/a	–
Supervisor + Hybrid	–	–	–	n/a	–
Full system	–	–	–	–	–

System Metrics. Chat LLM-judge pass rate (threshold 0.55), tool-call path selection accuracy, argument validity, result correctness, end-to-end tool success rate, and reranker expected-source coverage.

4.4 Experimental Scenarios

We design three experimental scenarios, each corresponding to a distinct evaluation table.

Scenario 1: Retrieval Ablation This scenario isolates the contribution of each retrieval component by comparing four configurations on the same 100-question dataset. Table 5 presents the planned metrics.

This ablation will answer **RQ3** by quantifying whether hybrid retrieval with reranking outperforms single-mode approaches. The existing codebase provides `evaluate_ragas.py` and `score_ragas.py` for automated execution.

Scenario 2: Graph and Agent Ablation This scenario evaluates the knowledge graph integration and routing mechanism independently from document retrieval. Table 6 presents the planned metrics.

This ablation will address **RQ1** by comparing supervisor-based routing against fixed routing, and **RQ2** by evaluating whether IG-SRR lane filtering improves answer quality.

Scenario 3: Full System Evaluation This scenario reports the end-to-end system performance using the complete CTU-Chat pipeline. Table 7 presents results from the current implementation.

Table 7: Scenario 3: Full system evaluation—recorded implementation metrics.

Evaluation	Num./Denom.	Result Interpretation
Chat judge pass rate	91/100	91.00% Latest 100-question run
Chat mean judge score	–	81.35% LLM judge score
Chat API errors	0/100	0 HTTP/API layer
Tool path selection	30/30	100.00% Three tool families
Tool argument validity	29/30	96.67% One argument mismatch
Tool result correctness	30/30	100.00% Deterministic checks
Tool end-to-end cases	29/30	96.67% Isolated sessions
Reranker source hit	80/80	100.00% All three reranker variants
<i>Domain breakdown (pass rate):</i>		
Exemption/Social support	25/25	100%
Tuition	23/25	92%
Scholarship	23/25	92%
Student loans	20/25	80%

The full system achieved 91% overall pass rate with 100% expected-source coverage across all reranker variants. Tool-calling accuracy reached 96.67%, with the single failure attributed to argument selection in the scholarship calculation tool. Domain analysis reveals strongest performance on exemption/social-support questions (100%) and weakest on student-loan queries (80%), consistent with the corpus containing overlapping loan regulations that challenge semantic disambiguation.

4.5 Discussion

The implementation results demonstrate that the asymmetric multi-agent architecture with IG-SRR produces a functional system with high overall quality, but several observations warrant discussion.

Strengths. The separation of concerns—exact catalog lookups and arithmetic in structured tools, semantic retrieval for policy context, and graph queries for relational data—prevents the failure modes common in monolithic RAG systems. The 100% tool-call selection accuracy confirms that the typed tool abstraction successfully eliminates schema hallucination on Neo4j queries. The reranker A/B experiment shows that all three scoring variants maintain complete source coverage, validating the IG-SRR lane filtering approach.

Domain-specific Performance Variation. Student-loan queries underperformed other domains (80% vs. 92–100%). This pattern is consistent with a corpus containing multiple overlapping loan regulations from different years, where the current lane filter (domain-level only) cannot distinguish between active and superseded provisions as effectively as the more granular tuition lane filters.

Limitations.

- The 100-question benchmark uses an LLM judge rather than paired human evaluation, limiting the strength of accuracy claims.
- Source-hit ranking is not equivalent to answer faithfulness; a system can retrieve the correct source but still generate an incorrect answer.
- The corpus contains multiple document versions and inactive records; meta-data quality directly affects retrieval precision.
- Gap 4 (temporal degradation) remains unaddressed beyond basic `academic_year` filtering; bi-temporal graph support is needed for production deployment.
- The rewrite guard is heuristic, and session memory is bounded by list length rather than a learned memory policy.
- No statistically powered ablation with confidence intervals has been conducted; the planned experiments in Tables 5 and 6 will address this.

5 Conclusion

This paper presented CTU-Chat, a Vietnamese university counseling system that addresses three of the five critical research gaps in Agentic GraphRAG systems for education. The *Asymmetric Multi-Agent* architecture matches agent autonomy to domain complexity, reducing computational overhead on simple queries while preserving multi-step reasoning for complex ones. *Intent-Guided Subspace Retrieval Routing* (IG-SRR) couples intent classification with deterministic metadata lane filters, confining hybrid retrieval to ontologically correct document subspaces and improving retrieval precision. *Typed Knowledge Graph tools* eliminate schema hallucination by replacing raw Cypher generation with safe, parameterized Python functions.

Preliminary evaluation on a 100-question Vietnamese benchmark demonstrates 91% LLM-judge pass rate, 96.67% tool-calling accuracy, and 100% expected-source coverage. Domain analysis reveals strong performance across most query types, with student-loan queries identified as the primary area for improvement due to overlapping regulations in the corpus.

The central engineering insight is the value of *separation of concerns*: exact lookups and arithmetic belong to structured components, semantic and lexical retrieval complement each other within lane-bounded subspaces, and graph queries should be used where the question is inherently relational. This asymmetric design philosophy offers a principled alternative to the homogeneous multi-agent architectures prevalent in current literature.

Future Work. Immediate priorities include completing the controlled retrieval ablations (Table 5) and agent ablation experiments (Table 6), conducting paired human evaluation alongside LLM-judge scoring, implementing bi-temporal graph support for temporal knowledge management (Gap 4), and developing a comprehensive multi-agent evaluation framework with execution-grounded metrics (Gap 5). We also plan to extend the benchmark to cover academic-program queries and to deploy CTU-Chat in a production pilot with real student users.

References

1. Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems* **33**, 9459–9474 (2020)
2. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* **3**(4), 333–389 (2009)
3. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In: *Proc. 32nd International ACM SIGIR Conference*, pp. 758–759 (2009)
4. Edge, D., Trinh, H., Cheng, N., et al.: From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024)
5. Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations* (2023)
6. Ma, T., La, T.T., Le Huu, L.T., et al.: REBot: From RAG to CatRAG with semantic enrichment and graph routing. *arXiv preprint arXiv:2510.01800* (2025)
7. Nogueira, R., Jiang, Z., Lin, J.: Document ranking with a pretrained sequence-to-sequence model. In: *Findings of EMNLP* (2020)
8. KA-RAG: Integrating Knowledge Graphs and Agentic Retrieval. *Applied Sciences* **15**(23), 12547 (2024)
9. CyVerACT: An Agentic Cypher Translation Workflow over Knowledge Graphs. *ResearchGate* (2024)
10. An Empirical Study of Multi-Agent RAG for Real-World University Applications. *arXiv:2507.11272* (2025)
11. Benchmarking Multi-Agent Architectures. *LangChain Blog* (2024)

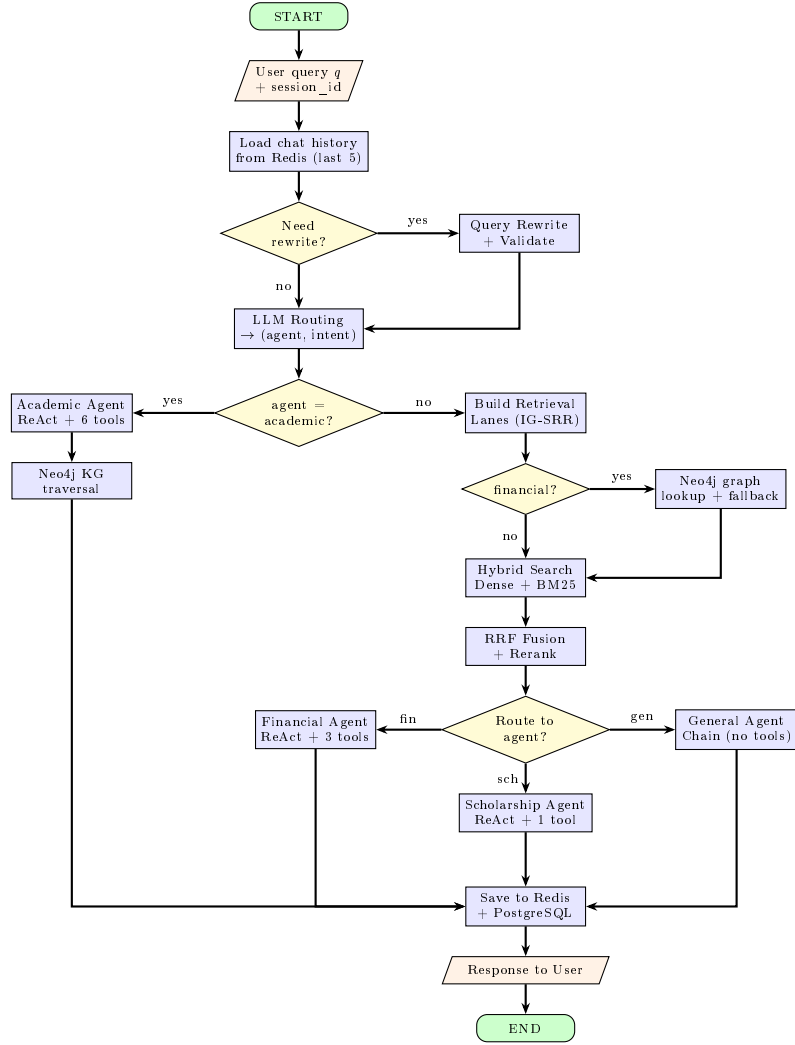


Fig. 2: Complete activity flowchart of CTU-Chat. Decision diamonds show branching logic: query rewrite validation, academic vs. retrieval routing, financial graph pre-lookup, and final agent dispatch.